



Universidade do Minho
Escola de Engenharia

Relatório 1^a Fase

Tomé Francisco Amorim Moreira - a111907

Miguel Pedro Ribeiro Correia - a111773

José Pedro Fernandes da Costa - a109807

16 de novembro de 2025

Índice Geral

1	Introdução	2
2	Sistema	2
3	Discussão	4
3.1	Query 1	5
3.2	Query 2	5
3.3	Query 3	6
3.4	Dados relativos ao programa-principal	8
3.5	Dados relativos ao programa-testes	8
4	Conclusão	11

1 Introdução

O trabalho prático teve como principal objetivo o desenvolvimento de um sistema para o processamento e análise de dados relativos a voos, aeroportos e restantes entidades envolvidas na gestão de um sistema de transporte aéreo.

Durante o desenvolvimento, privilegiámos uma estrutura baseada em módulos independentes, assegurando a modularidade e o encapsulamento do código desenvolvido.

Cada módulo foi responsável por uma parte específica do sistema — como gestão de entidades, estruturas de dados, parser, validação ou execução de queries. Paralelamente, foi dada especial atenção à eficiência do programa, tanto em termos de tempo de execução como de consumo de memória. Com este objetivo em mente recorremos a estruturas como tabelas de hash, filas com prioridade, árvores balanceadas, que permitem um acesso e manipulação rápida da informação mesmo para grandes volumes de dados.

Destaca-se o módulo de parsing genérico, concebido de forma adaptável a todas as entidades, permitindo uma leitura e validação uniforme dos dados sem duplicação de código.

2 Sistema

Optamos por uma divisão em módulos seguindo a seguinte estrutura:

- **Entidades** — optámos por isolar as diferentes entidades (como *Voo*, *Reserva*, *Passageiro*, *Aeronave* e *Aeroporto*) em módulos próprios, focando-nos nas operações de criação dos respetivos objetos, libertação de memória e definição dos seus *getters* e *setters*.

Considerámos esta divisão uma escolha natural, uma vez que cada entidade deve possuir um módulo que suporte tanto a sua construção como a sua manipulação de forma independente e encapsulada.

- **Gestores** — cada entidade possui um gestor individual, responsável por operações como a inserção de elementos nas respetivas estruturas de dados, manipulação dos dados e libertação dessas mesmas estruturas.

Cada um destes módulos foi desenvolvido com o objetivo de permitir uma resolução eficiente das *queries* propostas, assegurando uma boa separação de responsabilidades e um desempenho otimizado.

- **Validações** — foram desenvolvidos dois módulos dedicados à validação dos dados fornecidos.

O primeiro módulo é responsável pelas validações sintáticas, assegurando que o formato dos dados corresponde ao esperado. O segundo módulo realiza as validações lógicas, garantindo a consistência e integridade da informação ao longo do programa.

Esta separação foi adotada com o objetivo de manter a clareza e a organização do código, facilitando a manutenção e possíveis futuras alterações.

- **Parser** — foi desenvolvido para realizar uma leitura eficiente e genérica dos ficheiros CSV, garantindo a correta extração e organização dos dados antes de serem processados pelos restantes componentes do programa.

Optámos por isolar esta funcionalidade, uma vez que a leitura e interpretação dos ficheiros de entrada constituem uma parte crucial do funcionamento global do sistema.

- **Queries** — foram desenvolvidos três módulos distintos, cada um dedicado à resolução de uma *query* específica.

Em cada módulo encontram-se definidas as estruturas auxiliares e as estratégias necessárias para garantir uma execução eficiente e otimizada das operações associadas.

Esta abordagem modular permite não só uma melhor organização do código, como também uma maior clareza na lógica de resolução de cada *query*.

- **Estruturas de Dados** — desenvolvemos este módulo com a implementação de uma *priority queue*.

O objetivo deste módulo foi definir estruturas de dados não disponíveis na biblioteca *GLib*, que se revelaram essenciais para a resolução eficiente de algumas das *queries*, podendo ainda ser reutilizadas futuramente.

- **Interpretador** — responsável pela leitura e tratamento do ficheiro que contém as *queries* a serem resolvidas.

Este módulo realiza o *parsing* do ficheiro, identificando e executando cada *query* através da chamada às respetivas funções de resolução.

Optámos por esta abordagem para centralizar o controlo do fluxo do programa, garantindo uma separação clara entre a leitura dos comandos e a execução da lógica associada a cada *query*.

- **Testar** — responsável pela execução do programa de testes.

Esta divisão foi implementada com o objetivo de garantir uma separação clara entre o modo de testes e o modo de execução normal do programa, permitindo validar o correto funcionamento de cada uma das *queries*, bem como analisar o tempo de execução individual e global, e a memória utilizada.

- **Utils** — módulo essencial para todos os anteriormente mencionados, uma vez que disponibiliza um conjunto de funções de conversão de tipos.

Criámos este módulo com o intuito de ser reutilizado ao longo das mais variadas etapas de resolução do sistema, promovendo a reutilização do código.

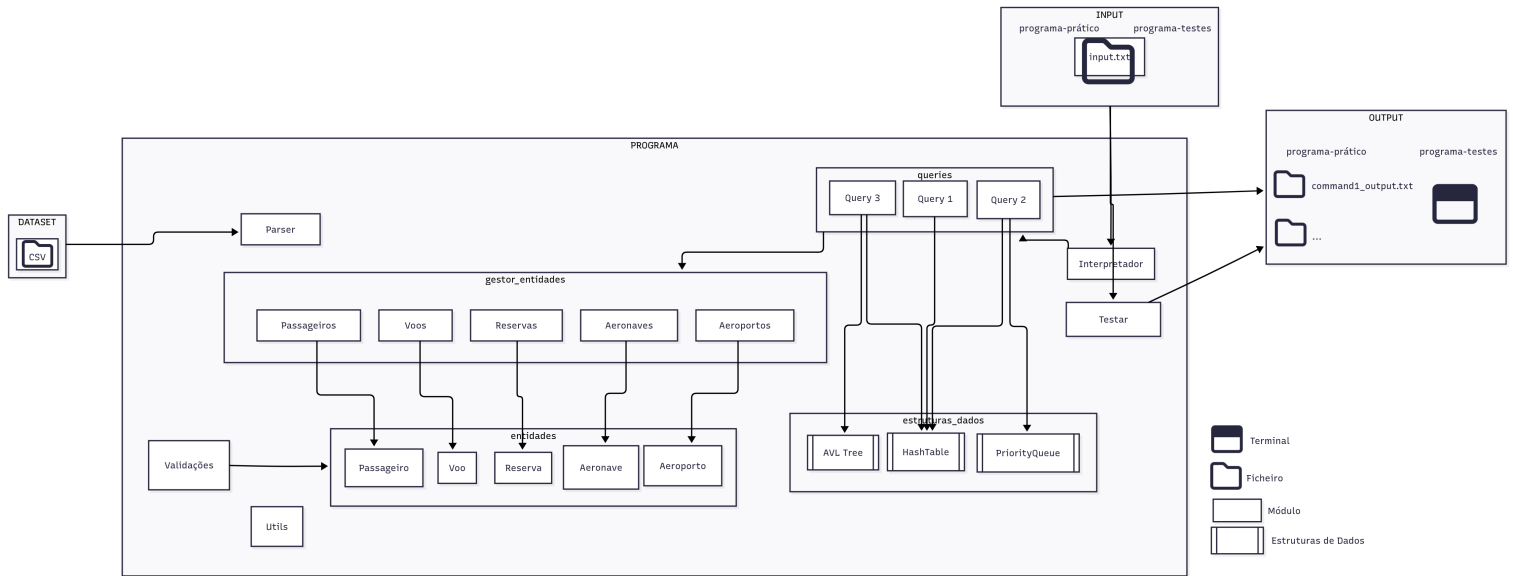


Figura 1: Arquitetura utilizada no programa

3 Discussão

Inicialmente, definimos todas as entidades que iriam ser utilizadas no projeto como **structs**, organizadas em módulos separados e opacos. Esta abordagem permitiu-nos ter uma visão clara da estrutura do sistema desde o início, garantindo a **modularidade** e o **encapsulamento** do código.

Em seguida, optámos por implementar primeiro as **funções de validação**, tanto sintáticas como lógicas. Esta decisão prendeu-se com o facto de, nesta fase inicial, ainda não termos definido detalhadamente a forma como seria feito o *parsing* nem a implementação das *queries*. Trabalhar nas validações desde cedo permitiu-nos estabelecer mecanismos de controlo de consistência e integridade dos dados antes de avançarmos para os módulos mais complexos do projeto.

Após a conclusão das funções de validação, iniciámos o desenvolvimento do módulo de *parsing*, com o objetivo de realizar a leitura dos ficheiros CSV. Durante esta fase, verificámos que as funções de *parsing* para cada entidade eram bastante semelhantes, o que nos levou à criação de uma função de *parsing* genérica, aplicável a todos os objetos do projeto.

Esta solução aumentou a reutilização de código e simplificou a manutenção, ao permitir distinguir o tipo de objeto e a estrutura onde este seria inserido através de funções específicas passadas como argumento.

Os dados foram armazenados em hash tables, o que nos permitiu aceder rapidamente às entidades durante a execução das queries. Esta escolha revelou-se eficiente para o tipo de operações exigidas nesta fase.

Quando decidimos que já tínhamos uma estrutura base para o nosso programa, passámos à implementação das queries.

3.1 Query 1

Para a implementação desta query, como todas as entidades já se encontravam armazenadas em **hash tables**, considerámos mais intuitivo realizar a busca diretamente através de um *lookup* na tabela utilizando como **key** o código IATA do aeroporto. Em todas as operações de acesso aos dados, recorreremos aos *getters* dos campos das entidades, garantindo o respeito pelo princípio do **encapsulamento** do código.

Análise assintótica

Como o acesso à hash table é feito em tempo constante, a complexidade da Query 1 é:

$$O(1)$$

3.2 Query 2

Durante o desenvolvimento da *query*, foram consideradas diversas abordagens para a estruturação e o acesso eficiente aos dados. Inicialmente, pensou-se em manter um array separado para cada fabricante, o que permitiria um agrupamento direto das aeronaves. Contudo, essa solução implicava em percorrer toda a hash table global para atualizar ou consultar as informações de cada fabricante, o que tornava o processo menos eficiente à medida que o número de fabricantes aumentava.

Outra alternativa estudada foi o uso de uma max-heap (fila de prioridade) para manter as aeronaves ordenadas, de forma que as com maior número de voos pudessem ser rapidamente recuperadas. No entanto, essa abordagem também exigia percorrer toda a hash table para alimentar a heap, o que resultava em sobrecarga adicional e complexidade desnecessária.

A solução final adotada foi a implementação de uma lista por fabricante, onde cada fabricante mantém sua própria coleção de aeronaves, acompanhada de um índice interno (hash table) para acesso rápido por identificador. Essa estrutura permite aplicar filtros diretos (por fabricante) com acesso imediato às aeronaves desejadas, sem a necessidade de percorrer todo o conjunto global.

Nos casos em que não há filtro de fabricante, as listas individuais podem ser mescladas de forma eficiente através de uma priority queue (max-heap), garantindo a ordenação global com bom desempenho.

Para possibilitar essa ordenação, foi introduzido um novo campo na entidade **Aeronave**, representando o número de voos (**flight_count**). A ordenação das listas é realizada uma única vez, logo após a leitura de todos os voos, minimizando o custo de processamento durante as consultas.

Embora essa abordagem implique em um tempo de carregamento inicial ligeiramente maior – já que as listas são montadas e ordenadas no momento da leitura – o ganho em eficiência na execução das queries subsequentes é significativo. O uso adicional de memória é considerado aceitável, uma vez que compensa amplamente em desempenho, proporcionando uma execução mais fluida tanto nas consultas com filtro quanto nas sem filtro.

Análise assintótica

Para a Query 2, consideremos os dois cenários: com filtro (por fabricante) e sem filtro (top-N global).

- **Query com filtro (por fabricante):** Cada fabricante possui a sua lista de aeronaves previamente ordenada pelo número de voos. Assim, basta aceder diretamente aos N primeiros elementos da lista correspondente.

Complexidade total da query:

$$O(N)$$

- **Query sem filtro (top-N global):** As listas já estão ordenadas por fabricante. Para obter os N maiores elementos globalmente, utiliza-se um *k-way merge* com uma *heap* de tamanho m (número de fabricantes). O processo consiste em:
 - $O(m) \rightarrow$ inicializar a heap com o primeiro elemento de cada fabricante;
 - $O(N \log m) \rightarrow$ extrair os N maiores elementos, mantendo a heap.

Complexidade total da query:

$$O(m + N \log m)$$

3.3 Query 3

Para a implementação desta query, decidimos utilizar uma **AVL Tree** para armazenar as datas dos voos, o que facilita a consulta eficiente entre duas datas solicitadas. A árvore está organizada da seguinte forma:

- **Esquerda** $\rightarrow$ datas mais antigas;
- **Direita** $\rightarrow$ datas mais recentes.

Paralelamente, criámos uma **hash table** utilizando o código IATA do aeroporto como chave, associando cada aeroporto apenas ao número de voos realizados (posteriormente detalhado por data).

Para cada solicitação da Query 3, realizamos um percurso na árvore entre $[data_inicial, data_final]$. Durante este percurso:

- Atualizamos simultaneamente a **hash table global**, mantendo em tempo real o **melhor aeroporto** e o respectivo número de voos;
- Desta forma, evitamos no final ter de percorrer toda a hash table para determinar o melhor aeroporto.

Essa abordagem garante que, ao percorrer a árvore apenas no intervalo de datas solicitado, conseguimos simultaneamente atualizar e manter a informação global do melhor aeroporto, de forma eficiente.

Análise assintótica

Seja n o número total de voos na AVL Tree, k o número de voos no intervalo $[data_inicial, data_final]$, e A o número de aeroportos distintos.

- Percurso na AVL Tree: $O(\log n + k)$, onde $\log n$ é o custo para localizar o intervalo inicial, e k o número de voos percorridos no intervalo.
- Atualização da hash table global: $O(1)$ por voo, totalizando $O(k)$ para o intervalo.
- Determinar o melhor aeroporto ao longo do percurso: $O(1)$ por voo, pois é atualizado durante a inserção na hash table.

Complexidade total da query:

$$O(\log n + k)$$

3.4 Dados relativos ao programa-principal

Etapa	Tempo (s)
Parsing	7,967
Validação dos dados	5,051
Execução de queries	1,496
Escrita em ficheiros	0,000
Total	14,514

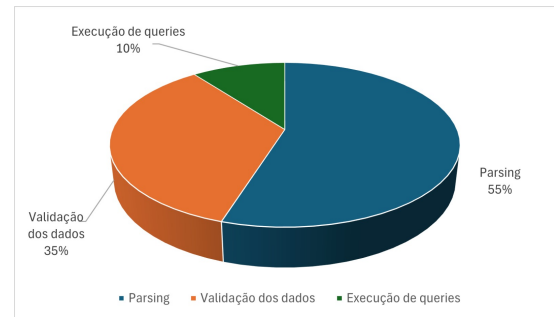


Figura 2: Tabela e gráfico referentes ao programa-principal com dataset sem erros

Etapa	Tempo (s)
Parsing	11,172
Validação dos dados	6,850
Execução de queries	1,490
Escrita em ficheiros	0,261
Total	19,773

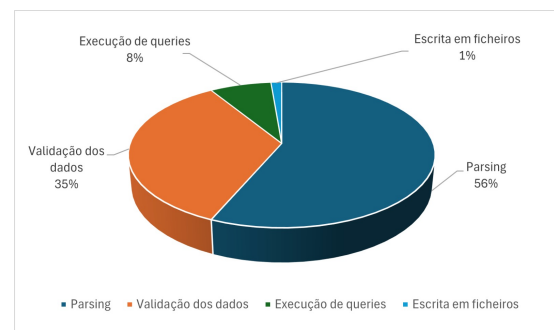


Figura 3: Tabela e gráfico referentes ao programa-principal com dataset com erros

Utilizando o ficheiro `input.txt` e os datasets fornecidos, obtivemos os tempos de execução apresentados nas figuras.

A análise permite concluir que, quer os dados contenham erros, quer não, não existem diferenças significativas no tempo de execução das funções de validação nem nas queries.

A maior sobrecarga ocorre no módulo de **parsing**, devido à necessidade de gerir corretamente a memória — nomeadamente a libertação de objetos em caso de erro lógico. Adicionalmente, a escrita no ficheiro de erros também contribui para o aumento do tempo de processamento, embora de forma relativamente pequena.

Em suma, o sistema mantém uma execução eficiente das queries, independentemente da presença de erros, sendo o impacto mais relevante limitado ao parsing e ao registo de inconsistências.

3.5 Dados relativos ao programa-testes

Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!
Memória utilizada: 599 MB Tempos de execução: Q1: 0.015 ms Q2: 0.019 ms Q3: 20.677 ms Tempo total: 0.830 s	Memória utilizada: 599 MB Tempos de execução: Q1: 0.067 ms Q2: 0.031 ms Q3: 31.903 ms Tempo total: 1.283 s	Memória utilizada: 599 MB Tempos de execução: Q1: 0.066 ms Q2: 0.025 ms Q3: 23.426 ms Tempo total: 0.943 s	Memória utilizada: 599 MB Tempos de execução: Q1: 0.066 ms Q2: 0.029 ms Q3: 24.255 ms Tempo total: 0.976 s

Resultados Teste 1:

Média de memória usada: 599 MB
Tempo médio de execução (Q1): 0,054 ms
Tempo médio de execução (Q2): 0,026 ms
Tempo médio de execução (Q3): 25,065 ms
Tempo médio de execução (Testes): 1,008 s
Especificações: Ubuntu 24.04.3 LTS, Intel Core i5-13420H × 12, 16 GB RAM, RTX 4050 Laptop GPU.

Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!
Memória utilizada: 599 MB Tempos de execução: Q1: 0.009 ms Q2: 0.012 ms Q3: 16.090 ms Tempo total: 0.658 s	Memória utilizada: 600 MB Tempos de execução: Q1: 0.234 ms Q2: 0.014 ms Q3: 14.515 ms Tempo total: 0.591 s	Memória utilizada: 600 MB Tempos de execução: Q1: 0.236 ms Q2: 0.015 ms Q3: 14.425 ms Tempo total: 0.588 s	Memória utilizada: 600 MB Tempos de execução: Q1: 0.232 ms Q2: 0.015 ms Q3: 14.495 ms Tempo total: 0.590 s

Resultados Teste 2:

Média de memória usada: 600 MB
Tempo médio de execução (Q1): 0,178 ms
Tempo médio de execução (Q2): 0,014 ms
Tempo médio de execução (Q3): 14,881 ms
Tempo médio de execução (Testes): 0,607 s
Especificações: Ubuntu 24.04.3 LTS, Intel Core i5-1235U × 12, 16 GB RAM, Intel Iris Xe Graphics (ADL GT2).

Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!	Q1: 40 de 40 testes ok! Q2: 40 de 40 testes ok! Q3: 40 de 40 testes ok!
Memória utilizada: 600 MB Tempos de execução: Q1: 0.061 ms Q2: 0.031 ms Q3: 11.385 ms Tempo total: 0.460 s	Memória utilizada: 600 MB Tempos de execução: Q1: 0.146 ms Q2: 0.039 ms Q3: 11.586 ms Tempo total: 0.472 s	Memória utilizada: 600 MB Tempos de execução: Q1: 0.061 ms Q2: 0.058 ms Q3: 12.692 ms Tempo total: 0.513 s	Memória utilizada: 600 MB Tempos de execução: Q1: 0.058 ms Q2: 0.042 ms Q3: 11.942 ms Tempo total: 0.483 s

Resultados Teste 3:

Média de memória usada: 600 MB
Tempo médio de execução (Q1): 0,082 ms
Tempo médio de execução (Q2): 0,043 ms
Tempo médio de execução (Q3): 11,901 ms
Tempo médio de execução (Testes): 0,482 s
Especificações: Ubuntu 24.04.3 LTS, Intel Core i9-13900HX × 8 (VM), 16 GB RAM, Hyper-V Virtual VGA (RTX 4070 Laptop GPU no host).

Figura 4: Resultados dos testes

Observando os tempos de execução, é possível notar que as *query 1* e *query 2* apresentam tempos significativamente reduzidos, consequência direta das estratégias previamente discutidas no tópico anterior.

Por outro lado, a *query 3* destaca-se das restantes por ser substancialmente mais lenta. Isto deve-se ao facto de que, para cada *query* deste tipo, é necessária a travessia da *AVL Tree* ao longo de todo o intervalo de datas especificado. Assim, quando o intervalo $[data_inicial, data_final]$ é grande, o tempo de execução é naturalmente penalizado.

Apesar disso, é possível que exista alguma estratégia adicional que permita melhorar estes tempos de execução, embora não tenhamos identificado nenhuma abordagem concreta até ao momento.

4 Conclusão

O desenvolvimento deste projeto permitiu-nos aplicar de forma prática conceitos fundamentais de programação modular, e também aprender a fazer um balanço entre a gestão de memória e a eficiência dos algoritmos utilizados. Pensamos também que a implementação do código com foco na **modularidade** e no **encapsulamento** demonstrou-se ser essencial para garantir um código organizado, legível e de fácil manutenção, o que eventualmente poderá ter um papel importante na resolução da fase 2 do projeto.

Uma das partes mais bem desenvolvidas do projeto foi o **parser genérico**, cuja implementação extensível simplificou significativamente o processo de leitura dos dados. Esta abordagem demonstrou as vantagens da reutilização de código, reduzindo a probabilidade de erros e facilitando futuras expansões na segunda fase.

Ao longo do desenvolvimento, procurou-se sempre encontrar um equilíbrio entre eficiência temporal e consumo de memória. Em vez de recorrer à alocação de memória estática, optámos pela utilização de estruturas dinâmicas, como *hash tables*, *priority queues* e árvores balanceadas, o que permitiu um desempenho consistente mesmo perante os grandes volumes de dados apresentados, ainda que à custa de um uso de memória ligeiramente superior, mas assegurando que o programa se mantém funcional e eficiente para datasets maiores.

Apesar dos resultados positivos, identificámos alguns pontos que poderão ser otimizados em iterações futuras. Um deles prende-se com a utilização excessiva da função **strdup**, o que aumentou o consumo de memória. Outro aspeto a melhorar é a gestão da memória durante o processo de validação — especialmente nos casos em que objetos são criados antes da validação lógica e libertados posteriormente em caso de erro.

Em suma, a primeira fase do projeto proporcionou uma aprendizagem sólida sobre a importância de projetar sistemas modulares, eficientes e bem estruturados. O balanço entre desempenho e utilização de recursos revelou-se também como uma aprendizagem importante.